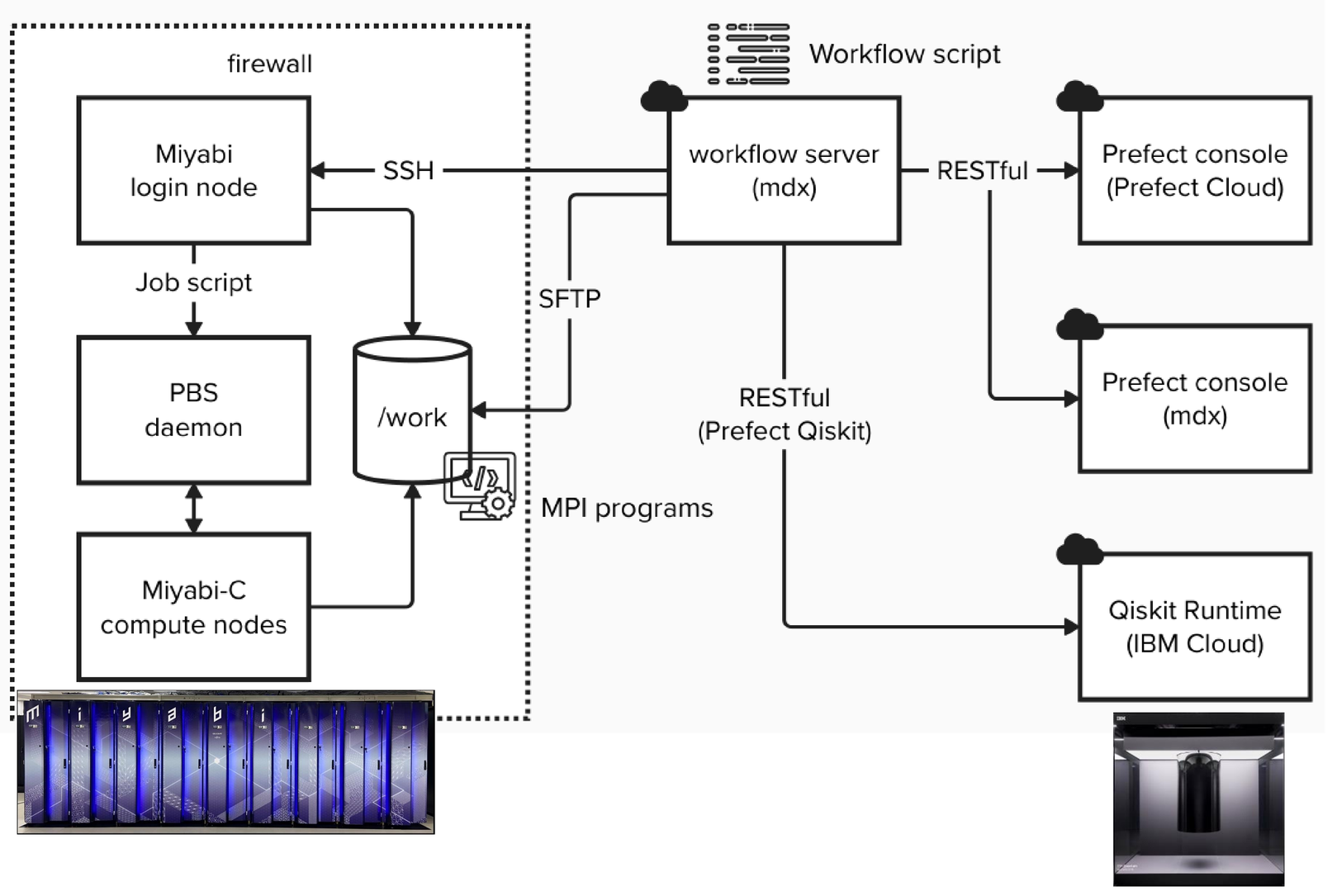
Prefect Tutorial for Miyabi

Hitomi Takahashi
IBM Quantum

Hybrid Application Development with Prefect Qiskit for QII members

- Run a workflow server outside HPC resources (a MDX server)
  - Submit Quantum jobs
  - Submit HPC jobs
  - Wait for results of jobs independently

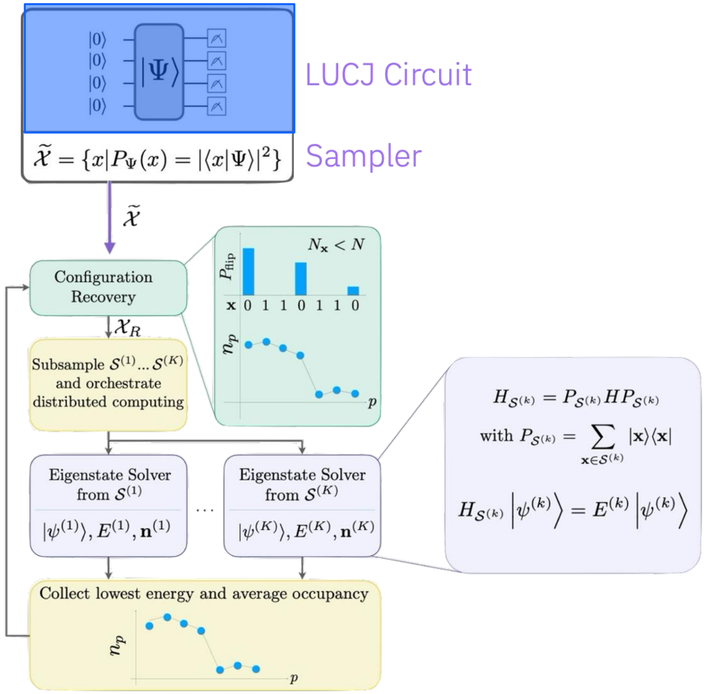
- Use Python to write workflow
  - Prefect
  - Prefect Qiskit to submit quantum jobs
  - qii-miyabi-Kawasaki to submit HPC jobs
    - Predefined jobs: SQD-Dice, SQD-SBD

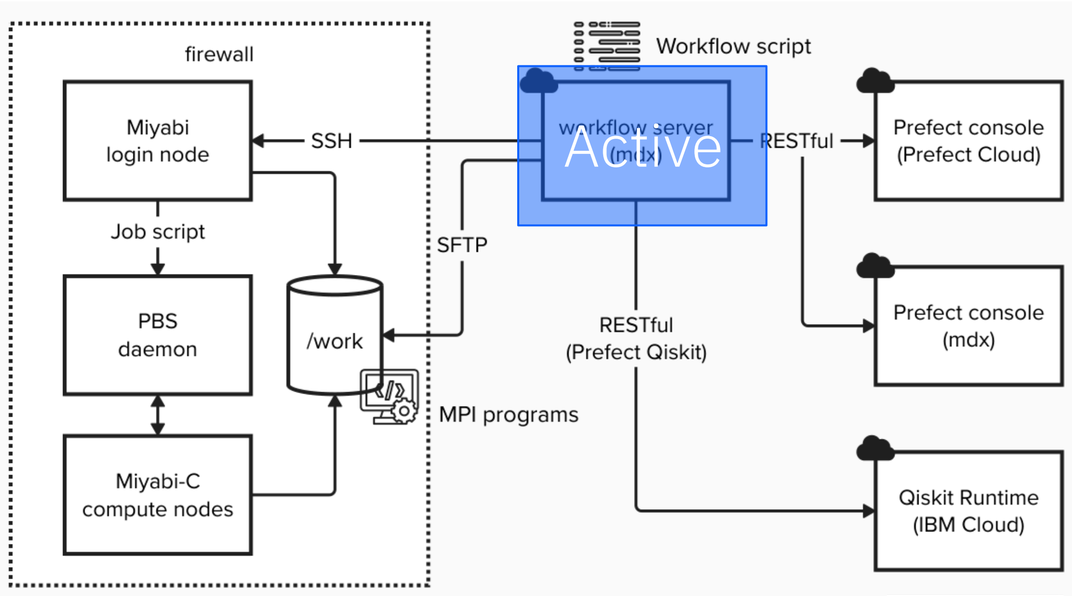


© 2026 IBM Corporation

What Is a QCSC Workflow?

- Quantum and classical (HPC) computations are tightly coupled
- Quantum computation is inherently probabilistic and requires repeated executions
- Experiments are not completed with a single run

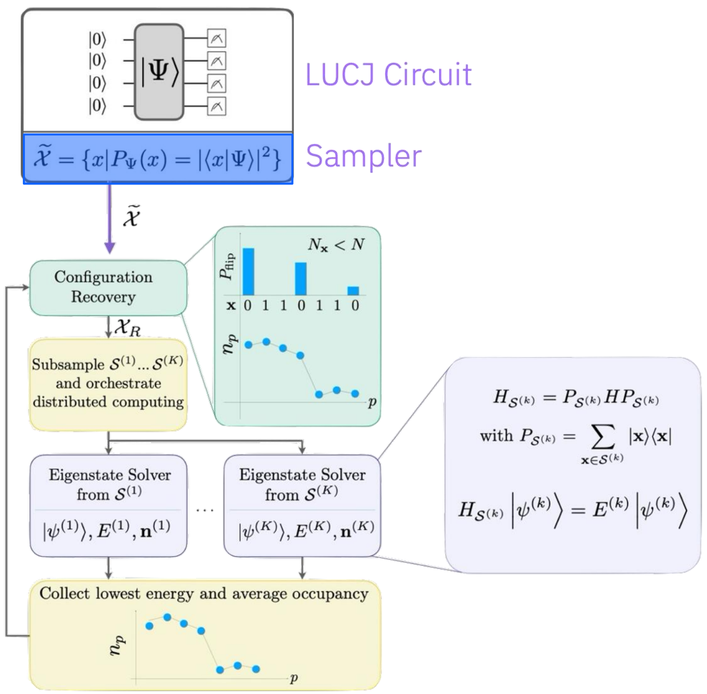


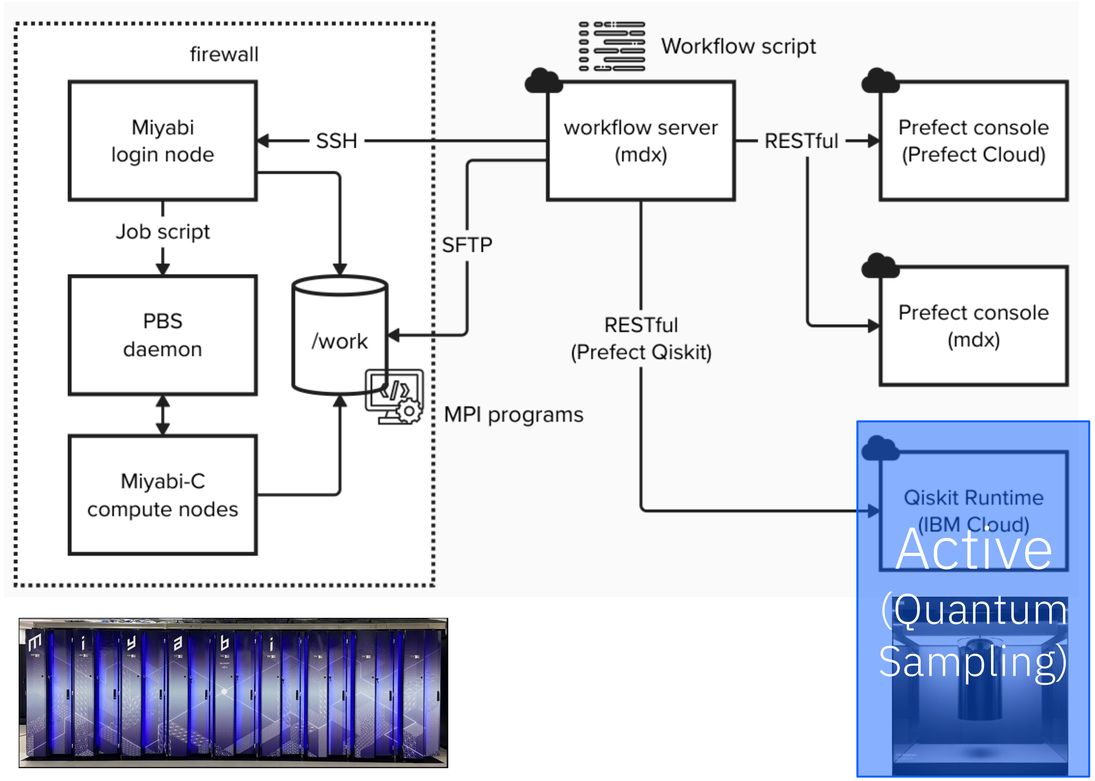


© 2026 IBM Corporation

What Is a QCSC Workflow?

- Quantum and classical (HPC) computations are tightly coupled
- Quantum computation is inherently probabilistic and requires repeated executions
- Experiments are not completed with a single run

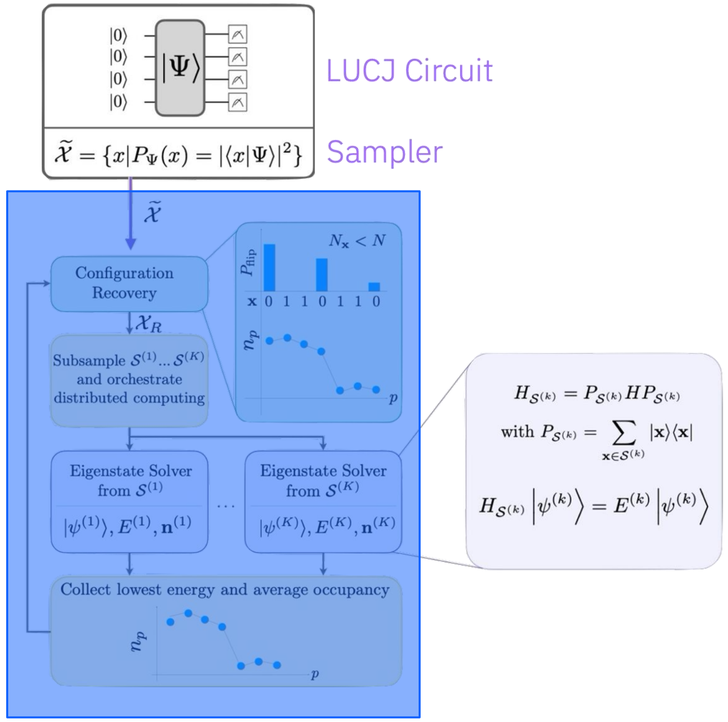


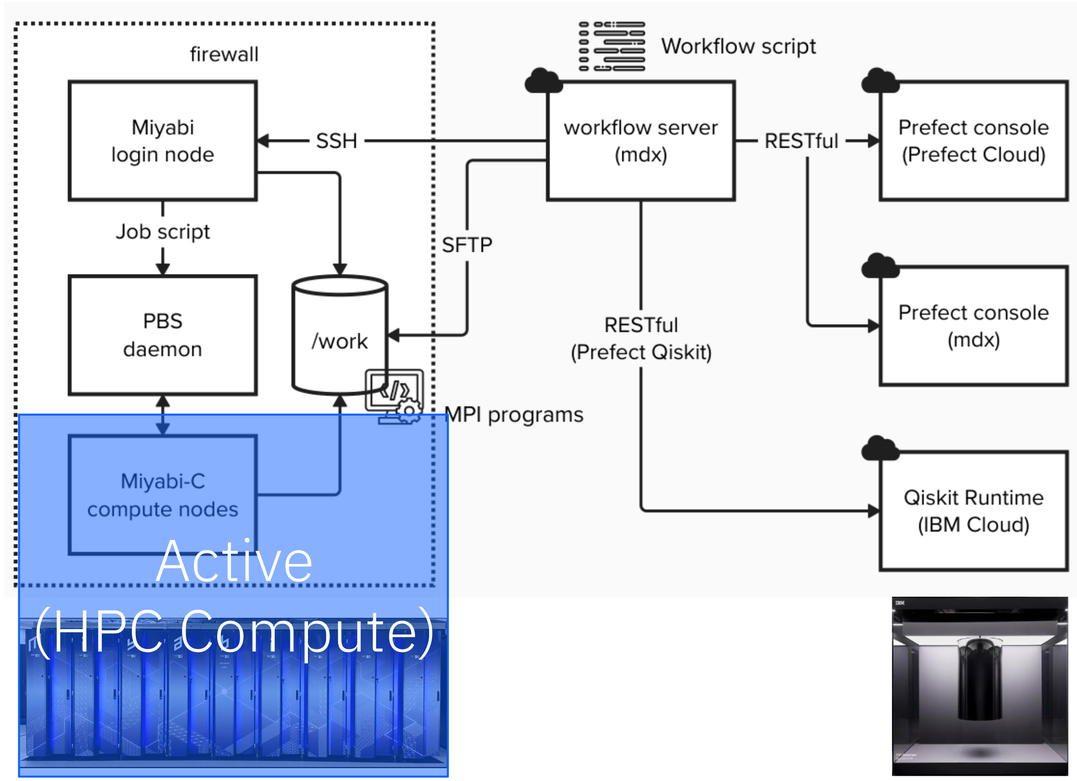


© 2026 IBM Corporation

What Is a QCSC Workflow?

- Quantum and classical (HPC) computations are tightly coupled
- Quantum computation is inherently probabilistic and requires repeated executions
- Experiments are not completed with a single run





© 2026 IBM Corporation

Why Scripts Are Not Enough for QCSC Workflow?

The problems for typical script-based approach

Execution history is not systematically recorded

It becomes unclear which parameters were used and when jobs were executed.

Recovery from failures is difficult

A failure in the middle often requires restarting the entire workflow.

Quantum and classical steps are fragmented

Quantum executions, HPC jobs, and post-processing are managed separately.

Reproducibility and comparison are hard

It is difficult to trace why a particular result was obtained.

© 2026 IBM Corporation

How Prefect Changes This

Applying a Workflow Engine to QCSC with Prefect

Visible execution and failures

Each step's status and logs are visible, making it clear where and why a run failed.

Efficient recovery from failures

Only failed tasks need to be re-executed, avoiding unnecessary re-runs of expensive computations.

Unified quantum–classical workflows

Quantum executions and HPC jobs are handled within a single, coherent workflow.

Reproducible experimental runs

Results are automatically linked with parameters and execution context, enabling reliable comparison.

© 2026 IBM Corporation

Prefect Core Concepts

Flow (end-to-end workflow)

- The Flow represents the full QCSC iteration

Tasks (individual execution steps)

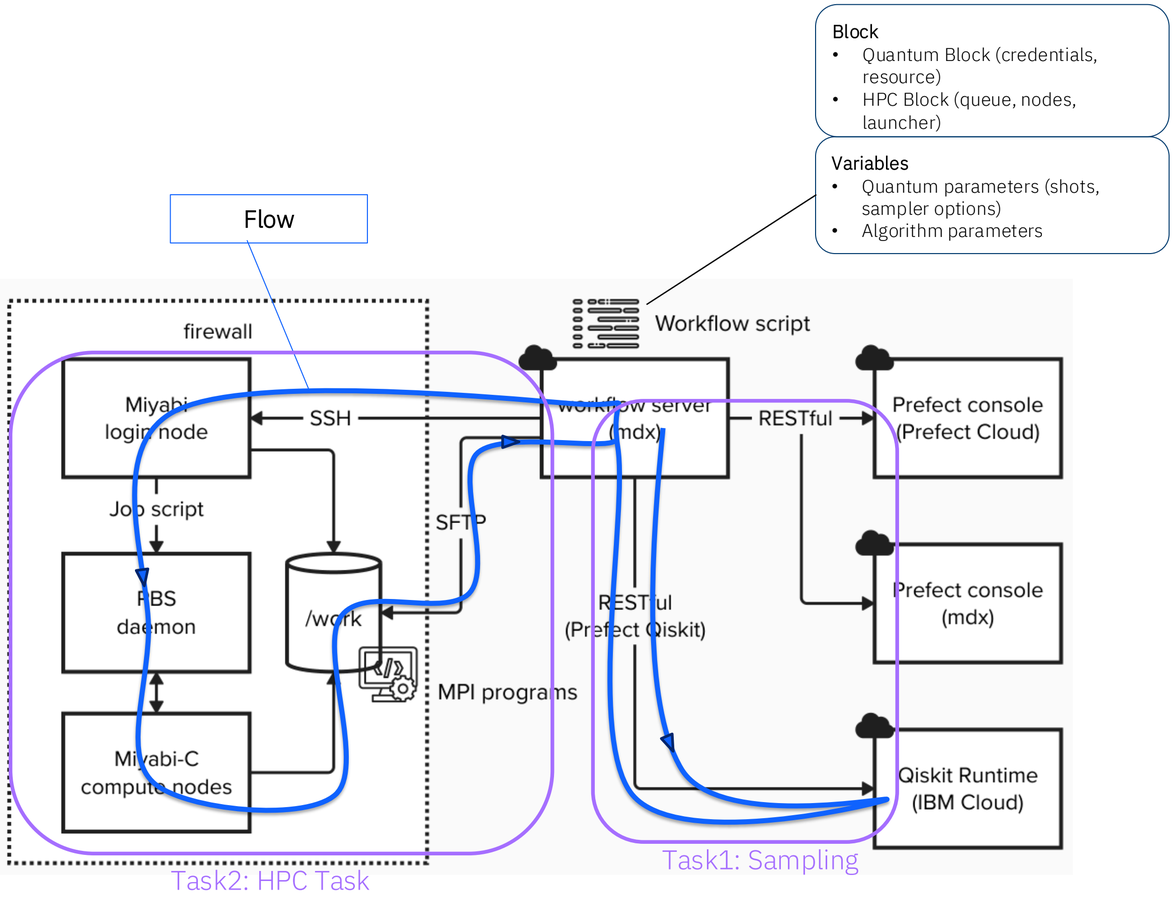
- Sampling Task: run the Sampler.
- HPC Task: submit a PBS job to run MPI programs.

Blocks (reusable configuration + credentials)

- Quantum Blocks: IBM Quantum credentials / runtime configuration.
- HPC Blocks: PBS job configuration.

Variables (run-time parameters)

- Quantum parameters: shots, circuit or sampler options.
- Algorithm parameters: iteration used by the workflow logic.



© 2026 IBM Corporation

From Workflow Concepts to Portable HPC Execution

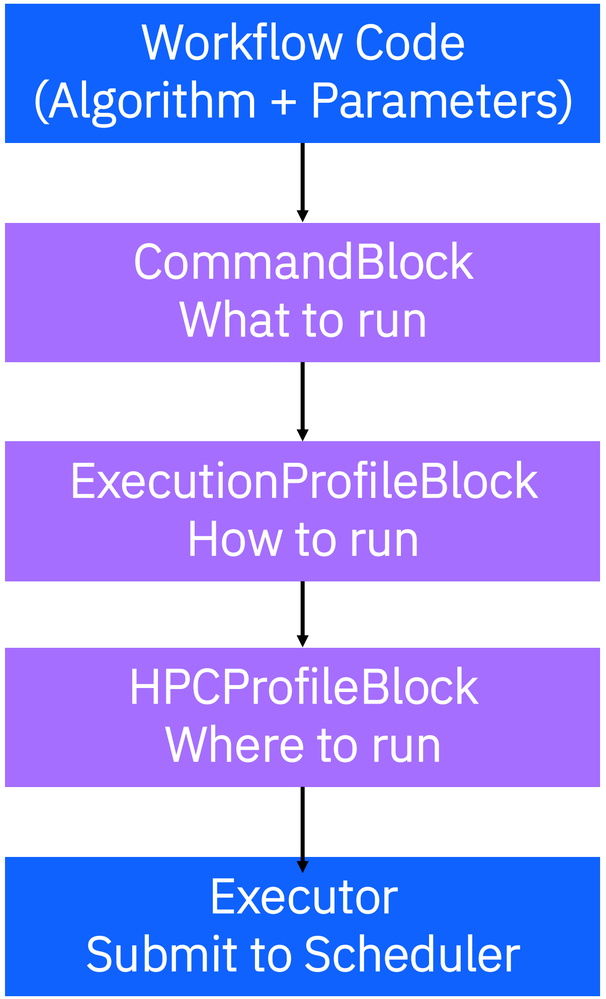
So far: why workflow engines matter

Next: how HPC-Prefect encodes HPC differences in code

Goal: run the same workflow logic on Miyabi and Fugaku

Separate Intent from Environment

- CommandBlock = WHAT to run
- ExecutionProfileBlock = HOW to run
- HPCProfileBlock = WHERE to run



Switch HPC Target Without Rewriting Workflow Logic

- Workflow logic stays the same.
- Main switch point: `hpc_profile_block_name`
- `script_filename` follows scheduler convention (.pbs / .pjm)

```
# Miyabi
await run_job_from_blocks(
    command_block_name="cmd-simulation",
    execution_profile_block_name="exec-simulation-mpi",
    hpc_profile_block_name="hpc-miyabi",
    work_dir="./work",
    script_filename="simulation.pbs",
)

# Fugaku
await run_job_from_blocks(
    command_block_name="cmd-simulation",
    execution_profile_block_name="exec-simulation-mpi",
    hpc_profile_block_name="hpc-fugaku",
    work_dir="./work",
    script_filename="simulation.pjm",
)
```

IBM

Backup

What a Workflow Engine Provides?

Role of a Workflow Engine

Explicit execution flow

The execution order and dependencies between steps are clearly defined and managed by the system.

Execution visibility

The status of each step, including failures, can be observed in a unified view.

Recorded history and parameters

Execution results, parameters, and metadata are automatically stored for later analysis and comparison.

Safe re-execution

Failed steps can be retried or re-run without restarting the entire workflow.